



INFORMATIKA ÉS TÁVKÖZLÉS ÁGAZAT

Project Ascension

Készítette

Rajna Torda Imre

Müller Zsolt István

Csillik Gergely

Szoftverfejlesztő és -tesztelő szak

Témavezető

Horvath Istvan

oktató

MISKOLC, 2026

Szoftverfejlesztő és -tesztelő szak

Dokumentáció

tudnivalók a záródolgozathoz

Rajna Torda Imre
Müller Zsolt István
Csillik Gergely

2025–2026

Tartalomjegyzék

1. Bevezetés	1
1.1. A projekt bemutatása és célja	1
2. Követelményanalízis	2
2.1. Funkcionális követelmények	2
2.2. Nem funkcionális követelmények	3
2.3. Használati esetek (Use Cases)	3
3. Fejlesztői környezet	5
3.1. Fejlesztési környezet és technikai háttér	5
3.2. Projektstruktúra és komponensek szerepe	5
3.3. Telepítési és indítási folyamat	6
3.4. Fejlesztési workflow és napi munkamenet	8
3.5. Hibakeresési módszertan és diagnosztika	8
3.6. Tesztelési környezet és minőségbiztosítási gyakorlat	8
3.7. Környezeti változók és konfiguráció	9
3.8. Tipikus hibák és elhárításuk	9
3.9. Karbantarthatóság és továbbfejlesztési szempontok	10
3.10. Összegzés	10
4. Vizsgaremek és funkcióinak bemutatása	11
4.1. Bejelentkezés és regisztráció	11
4.2. Dashboard áttekintő	11
4.3. Test modul: étkezés és edzés	12
4.4. API dokumentáció (Swagger)	13
5. Rendszertervezés	14
5.1. Adatmodell és architektúráis kapcsolatok	14
5.2. REST API tervezési összefoglaló	15
5.3. Felületi és navigációs tervezési elvek	15
5.4. Tervezési döntések indoklása	15

6. Megvalósítás	16
6.1. Adatbázis réteg implementációja	16
6.2. Backend implementáció és üzleti logika	17
6.3. Biztonság, jogosultság és hibakezelés	17
7. Minőségbiztosítás és Tesztelés	18
7.1. Unit tesztek (Egységtesztelés)	18
7.2. API tesztelés (Postman/Swagger)	18
7.3. Felhasználói felület és Reszponzivitás tesztelése	19
7.4. Hibajegyzék és javítások	19
8. Felhasználói útmutató	20
8.1. Rendszerhasználat előfeltételei	20
8.2. Regisztráció és bejelentkezés	20
8.3. Modulok használata	21
8.4. Dashboard és profil	21
8.5. Tipikus felhasználói folyamat	21
8.6. Felhasználói hibák és rövid megoldások	22
9. Összegzés	23
9.1. Projekt értékelése	23
9.2. Továbbfejlesztési lehetőségek	23
Mellékletek	II
Irodalomjegyzék	III

1. fejezet

Bevezetés

1.1. A projekt bemutatása és célja

Az Ascension projekt létrehozását az indokolta, hogy az életmódváltás több területének (edzés, táplálkozás, napi szokások) párhuzamos követése sok felhasználó számára nehézkes, a különálló alkalmazások pedig nem adnak átfogó képet a fejlődésről. A fejlesztés célja egy egységes, könnyen használható webalkalmazás készítése volt, amely támogatja a tudatos önfejlesztést.

A rendszerben a felhasználó egyetlen felületen tud regisztrálni és belépni, kezelni profiladatait, naplózni étkezéseit és edzéseit, valamint visszajelzést kapni saját előrehaladásáról. Az elkészült megoldás egy működő, továbbfejleszthető alaprendszer, amelyre további funkciók biztonságosan építhetők.

A megvalósítás során a kliensoldali felület React alapokon készült, amely gyors és áttekinthető felhasználói interakciót biztosít. A serveroldali működést Node.js és Express alapú REST API valósítja meg, az adatkezelést pedig relációs adatbázis támogatja.

A dolgozat további része bemutatja a követelményeket, a tervezési döntéseket, a rendszerarchitektúrát és a megvalósítás fő lépéseit. Ezt követi a tesztelési és értékelési fejezet, amely a működés igazolását és a korlátok ismertetését tartalmazza. A záró rész összegzi az eredményeket, valamint kijelöli a lehetséges továbbfejlesztési irányokat.

2. fejezet

Követelményanalízis

2.1. Funkcionális követelmények

Az Ascension rendszer funkcionális követelményei a jelenlegi implementáció alapján:

- A rendszer biztosítsa a felhasználók regisztrációját és bejelentkezését (JWT alapú hitelesítéssel).
- A felhasználó tudja kezelni saját profiladatait (pl. életkor, testsúly, magasság, cél, aktivitási szint, edzéstapasztalat).
- A rendszer tegye lehetővé étkezési adatok rögzítését, listázását és törlését dátum szerinti szűréssel.
- A rendszer támogassa az edzésadatok rögzítését, listázását és törlését (gyakorlat, időtartam, szettek, ismétlések, súly).
- A rendszer biztosítsa az arcápolási rutin mentését és a napi rutin-követés funkciót.
- A dashboard felület jelenítsen meg összesített visszajelzéseket a napi/heti/havi aktivitásról.
- A frontend felület legyen reszponzív, és mobil/asztali nézetben egyaránt használható.
- A védett végpontok csak érvényes Bearer token birtokában legyenek elérhetők.
- A rendszer biztosítsa, hogy a felhasználó kizárólag a saját adatait tudja elérni és módosítani.

2.2. Nem funkcionális követelmények

Az Ascension rendszer nem funkcionális követelményei:

- **Rendelkezésre állás és stabilitás:** a backend szolgáltatás folyamatos működése mellett a hibák kezelése strukturált HTTP válaszkódokkal történjen (4xx, 5xx).
- **Teljesítmény:** a gyakori lekérdezések (profil, napi bejegyzések, dátumszűrt listák) normál terhelés mellett rövid válaszüddel működjenek.
- **Biztonság:** a jelszavak bcrypt hash formában tárolóddanak, a védett végpontok JWT tokenhez kötöttek legyenek.
- **Adatintegritás:** az adatbázis idegen kulcsokat és ON DELETE CASCADE szabályokat alkalmazzon a konzisztencia érdekében.
- **Reszponzivitás:** a felhasználói felület mobil és asztali nézetben is használható maradjon.
- **Karbantarthatóság:** a frontend komponens-alapú, a backend moduláris felépítésű legyen, elkülönített felelősségi körökkel.
- **Skálázhatóság:** az architektúra tegye lehetővé új modulok (pl. további egészségügyi mérők) hozzáadását jelentős áttervezés nélkül.
- **Dokumentáltság:** a REST API Swagger felületen keresztül dokumentáltan legyen elérhető (/swagger).
- **Kompatibilitás:** a rendszer modern böngészőkben (Chrome, Edge, Firefox) azonos funkcionális viselkedést biztosítson.

2.3. Használati esetek (Use Cases)

UC1: Regisztráció ♦ A felhasználó kitölti a regisztrációs űrlapot (felhasználónév, e-mail, jelszó), a backend validálja az adatokat, bcrypt hash készül, majd mentés történik a `users` táblába és token generálódik.

UC2: Bejelentkezés ♦ A felhasználó megadja az e-mail/felhasználónév és jelszó adatokat, backend oldali ellenőrzés és bcrypt compare után sikeres esetben JWT token kerül kiadásra, amely hozzáférést biztosít a védett végpontokhoz.

UC4: Élelmiszer követés ♦ A felhasználó ételt választ, gramm mennyiséget ad meg, a rendszer kiszámítja a tápértékeket, majd menti a bejegyzést a `food_entries` táblába, és frissíti a napi listát.

UC5: Edzés rögzítés ♦ A felhasználó rögzíti az edzéstípus, időtartam és opcionális terhelési adatokat, a rendszer menti azokat a `workout_entries` táblába, majd frissíti a haladási mutatókat.

További használati esetek összefoglalása ♦ A profiladatok mentése (`/api/profile/details`) során a személyes paraméterek frissülnek, amelyek hatással vannak az összesített visszajelzésekre. Az arcápolási rutin kezelése a rutin mentését és napi teljesítés követését támogatja (`skin_routines`, `skin_routine_tracking`). A profil- és statisztikanézet a hitelesített felhasználó saját adatait jeleníti meg, kijelentkezéskor pedig a kliensoldali token és session adatok törlésre kerülnek.

3. fejezet

Fejlesztői környezet

3.1. Fejlesztési környezet és technikai háttér

Az Ascension rendszer fejlesztése és tesztelése Windows 11 Pro (10.0.26200) operációs rendszeren történt, 11th Gen Intel Core i7-11700K processzorral és 31.81 GB memóriával. Ez a konfiguráció stabilan lehetővé tette a frontend, backend és adatbázis-szolgáltatás párhuzamos futtatását, valamint az automatizált tesztek ismételt végrehajtását.

A projekt megvalósítása során Node.js v24.12.0 és npm 11.6.2 környezetet használtunk a JavaScript alapú fejlesztési feladatokhoz. A verziókezelés Git 2.52.0 segítségével történt. A kliensoldali fejlesztésnél Vite alapú React projekt futott, amely gyors fejlesztői visszacsatolást biztosított. A frontend indításához Bun 1.3.11 is használatban volt, míg az opcionális admin asztali felülethez .NET SDK 7.0.401 környezet tartozott.

A rendszer adatkezelése relációs adatbázisra épül, az inicializálás a projekt gyökerében található database.sql állomány alapján történik. A fejlesztési környezetben az adatbázis létrehozása és importálása phpMyAdmin felületen keresztül történt, így a szükséges táblák és kapcsolatok reprodukálható módon állíthatók elő új gépen is.

A 3.1. ábra a fejlesztéshez használt adatbázis-környezetet mutatja. A képen látható állapot igazolja, hogy az adatbázis-szolgáltatás elérhető, és az alkalmazás működéséhez szükséges táblák létrehozása sikeresen megtörtént.

3.2. Projektstruktúra és komponensek szerepe

A forráskód szerkezetileg elkülöníti a fő felelősségi területeket. Az Ascension-Backend könyvtár tartalmazza a REST API-t, az üzleti logikát, az autentikációs folyamatot, az ORM modelleket, valamint a teszteket. Az Ascension-Frontend-React könyvtárban található a felhasználói felület, az oldalak közötti navigáció és a komponens-alapú klienslogika.



3.1. ábra. Futó fejlesztői adatbázis-környezet (phpMyAdmin és XAMPP Control Panel)

A backend központi eleme az Express alapú szerver, amely a hitelesített és publikus végpontokat is kiszolgálja. A frontend React komponenseken keresztül hívja ezeket a végpontokat, így a rétegek közötti kommunikáció API-központúan történik. Ez a szétválasztás karbantarthatósági szempontból előnyös, mert a felület és a szerveroldali logika egymástól függetlenül fejleszthető.

A 3.2. ábra jól mutatja a frontend, backend és kiegészítő komponensek szétválasztását. Ez a felépítés csökkenti az egymásra épülő módosítások kockázatát, és gyorsabb hibakeresést tesz lehetővé.

3.3. Telepítési és indítási folyamat

A fejlesztői indítás első lépése az adatbázis inicializálása. Ennek során a database.sql script futtatása létrehozza az alkalmazás működéséhez szükséges táblákat és indexeket.

A 3.3. ábra a backend indításának sikeres állapotát mutatja. A futó szolgáltatás biztosítja, hogy a REST API végpontok tesztelhetők és a kliensoldalról elérhetők legyenek.

A 3.4. ábra alapján a frontend fejlesztői szerver indulása után az alkalmazás helyi címen közvetlenül elérhető. Ez gyors visszacsatolást ad a felület- és funkcióváltoztatások ellenőrzéséhez.

Alapértelmezett fejlesztői futási címek:

- Backend: `http://localhost:3000`
- Frontend: `http://localhost:5173`

Asztal > vizsga_jo > AscensionMain >				
Rendezés ▾ Megtekintés ▾ ...				
Név	Módosítás dátuma	Típus	Méret	
.env	2026. 04. 05. 0:15	Fájlmappa		
.vscode	2026. 04. 05. 12:40	Fájlmappa		
Ascension-Admin-WPF	2026. 04. 05. 12:31	Fájlmappa		
Ascension-Backend	2026. 04. 05. 23:28	Fájlmappa		
Ascension-Frontend-React	2026. 04. 05. 13:23	Fájlmappa		
docs	2026. 04. 05. 22:46	Fájlmappa		
node_modules	2026. 04. 05. 13:07	Fájlmappa		
.env	2026. 03. 30. 18:23	ENV fájl	1 KB	
.gitignore	2026. 04. 05. 12:39	Git Ignore forrásfájl	1 KB	
AscensionMain.sln	2026. 04. 05. 12:24	Visual Studio Solu...	2 KB	
database	2026. 03. 30. 18:23	SQL forrásfájl	10 KB	
package	2026. 04. 05. 13:12	JSON forrásfájl	1 KB	
package-lock	2026. 04. 05. 13:07	JSON forrásfájl	71 KB	
server.js	2026. 04. 05. 13:12	JSFile	1 KB	

3.2. ábra. A projekt könyvtárstruktúrája

```

(.venv) PS C:\Users\Torda1\Desktop\vizsga_jo\AscensionMain> cd ascension-backend
(.venv) PS C:\Users\Torda1\Desktop\vizsga_jo\AscensionMain\ascension-backend> npm install

up to date, audited 203 packages in 904ms

32 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
(.venv) PS C:\Users\Torda1\Desktop\vizsga_jo\AscensionMain\ascension-backend> npm start

> ascension-backend@1.0.0 start
> node server.js

✔ Sequelizee kapcsolódva
Backend fut: http://localhost:3000

```

3.3. ábra. A backend indításának lépései

```

(.venv) PS C:\Users\Torda1\Desktop\vizsga_jo\AscensionMain> cd ascension-frontend-react
(.venv) PS C:\Users\Torda1\Desktop\vizsga_jo\AscensionMain\ascension-frontend-react> bun install
bun install v1.3.11 (af24e281)

Checked 157 installs across 189 packages (no changes) [1362.00ms]
(.venv) PS C:\Users\Torda1\Desktop\vizsga_jo\AscensionMain\ascension-frontend-react> bun dev
$ vite

VITE v8.0.3 ready in 189 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

```

3.4. ábra. A frontend indításának lépései

– API dokumentáció: <http://localhost:3000/swagger>

3.4. Fejlesztési workflow és napi munkamenet

A fejlesztés iteratív munkamenetben zajlott, ahol a frontend és backend feladatok rövid ciklusokban, egymással összehangoltan készültek. A napi folyamat jellemzően a nyitott hibák és funkciók áttekintésével indult, ezt követte a lokális környezet ellenőrzése, majd az érintett modulok célzott módosítása.

Minden módosítás után azonnali visszaellenőrzés történt. Backend oldalon a naplók és a végpontválaszok, frontend oldalon a funkcionális és vizuális viselkedés alapján validáltuk a változtatásokat. Ez a gyakorlat csökkentette a később észlelt regressziók kockázatát.

A verziókezelési folyamatban a módosítások tematikus egységek szerint kerültek rögzítésre. A funkciófejlesztések, hibajavítások és dokumentációs változtatások elkülönítése elősegítette a visszakövethetőséget és gyorsította a hibák izolálását.

3.5. Hibakeresési módszertan és diagnosztika

A hibakeresés három fő nézőpontból történt: alkalmazáslogika, API-kommunikáció és kliensoldali megjelenítés. Backend oldalon a státusz kódok, hibaüzenetek és futási naplók adták az elsődleges diagnosztikai információt, különösen autentikációs, validációs és adatkapcsolati hibák esetén.

API szinten a Swagger felület és a célzott HTTP-hívások segítették a kérdés-válasz szerkezet ellenőrzését. A vizsgálat nemcsak a sikeres válaszokra, hanem a hibás bemenetek kezelésére is kiterjedt, így a rendszer viselkedése szélső esetekben is ellenőrizhető maradt.

Frontend oldalon a böngésző fejlesztői eszközei biztosították a gyors visszajelzést. A hálózati napló, konzolhibák és renderelési állapot alapján jól elkülöníthető volt, hogy a hiba kliensoldali, API-oldali vagy adatforrási eredetű.

3.6. Tesztelési környezet és minőségbiztosítási gyakorlat

A projekt minőségbiztosítása több rétegben történt. Az egységtesztek a kritikus számítási logikát ellenőrizték, míg az integrációs tesztek valós HTTP-hívásokkal vizsgálták a fő API-folyamatokat. A tesztelés célja az volt, hogy az új funkciók bevezetése mellett a meglévő működés is stabil maradjon.

A manuális funkcionális tesztek a felhasználói utakra koncentráltak: hitelesítés, profilkezelés, étel- és edzésnaplózás, valamint dashboard visszajelzések. A kézi ellenőrzés különösen fontos volt azoknál a pontoknál, ahol a vizuális visszacsatolás vagy a több komponensen átívelő folyamat dominált.

A kombinált tesztelési megközelítés eredményeként a hibák többsége fejlesztési környezetben azonosítható és javítható volt, ami csökkentette a későbbi integrációs kockázatokat.

3.7. Környezeti változók és konfiguráció

A környezeti változók olyan konfigurációs kulcsok, amelyeket indításkor lehet megadni, így nem szükséges a forráskód módosítása. A jelenlegi implementációban az alábbi kulcsok használhatók:

- ADMIN_USERNAME
- ADMIN_PASSWORD
- ADMIN_JWT_SECRET
- FDC_API_KEY

Az ADMIN_USERNAME, ADMIN_PASSWORD és ADMIN_JWT_SECRET az admin hitelesítés paramétereit befolyásolja, az FDC_API_KEY pedig a külső élelmiszer-adatforráshoz tartozó hozzáférési kulcs. Fejlesztési és éles környezetben ezek elkülönített kezelése javasolt.

A konfigurációkezelés célja az volt, hogy a projekt lokális futtatása egyszerű maradjon, ugyanakkor az érzékeny beállítások ne legyenek kódba rögzítve. Ez a gyakorlat előkészíti a biztonságosabb éles környezeti átállást is.

3.8. Tipikus hibák és elhárításuk

Fejlesztés közben a leggyakoribb probléma a portütközés, amikor a 3000 vagy 5173 portot egy másik folyamat használja. Ilyenkor a folyamat leállítása vagy a port átállítása szükséges.

Szintén gyakori az adatbázis-kapcsolati hiba, amelyet legtöbbször a nem futó adatbázis-szolgáltatás, a hiányzó import vagy hibás elérési paraméter okoz. Az ilyen esetek a backend naplók alapján gyorsan diagnosztizálhatók.

Hitelesítési hibáknál tipikus a hiányzó vagy lejárt token, amely 401 vagy 403 választ eredményez. CORS-probléma esetén a kliens origin nincs engedélyezve a szerveren, ezért a kérés már böngészőszinten blokkolódik.

3.9. Karbantarthatóság és továbbfejlesztési szempontok

A rendszer architektúrája rétegesen elkülönített, így a frontend, backend és adatkezelési logika külön is módosítható. Ez csökkenti a komponensek közötti szoros függőséget, és megkönnyíti az új funkciók bevezetését.

A jelenlegi szerkezeti döntések támogatják a fokozatos bővítést: új modulok vagy kiegészítő végpontok beépítése úgy történhet, hogy a meglévő komponensek működése minimális kockázattal maradjon stabil.

A dokumentált fejlesztői folyamat, a képernyőképes alátámasztás és a tesztelési gyakorlat együtt olyan alapot ad, amely oktatási és projektátadási szempontból is megfelelő.

3.10. Összegzés

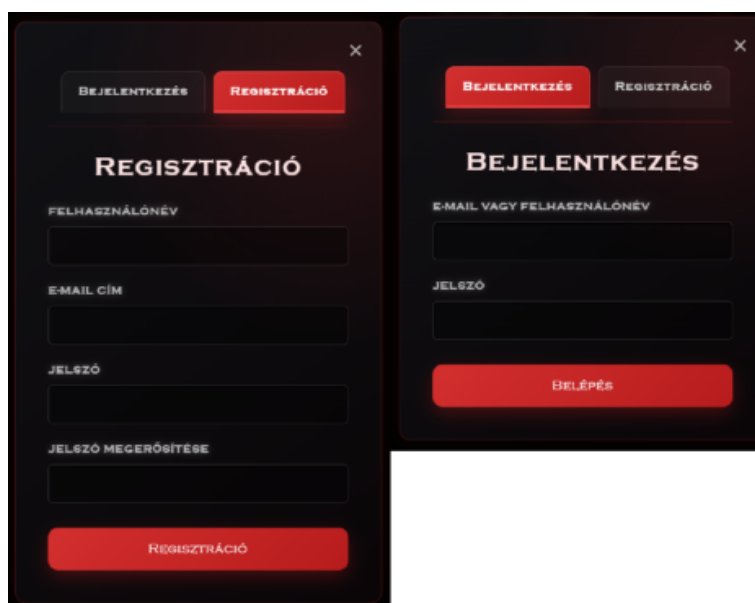
A fejlesztői környezet kialakítása során a cél egy stabil, reprodukálható és gyorsan indítható munkafolyamat biztosítása volt. A projekt jelenlegi felépítése alkalmas arra, hogy új modulokkal bővíthető maradjon, miközben a meglévő funkciók ellenőrizhetően és egységesen működnek.

A használt eszközkészlet, a rétegek szétválasztása, valamint a dokumentált indítási és hibakezelési folyamat együttesen olyan alapot ad, amely hosszabb távon is támogatja a karbantartható fejlesztést.

4. fejezet

Vizsgaremek és funkcióinak bemutatása

4.1. Bejelentkezés és regisztráció



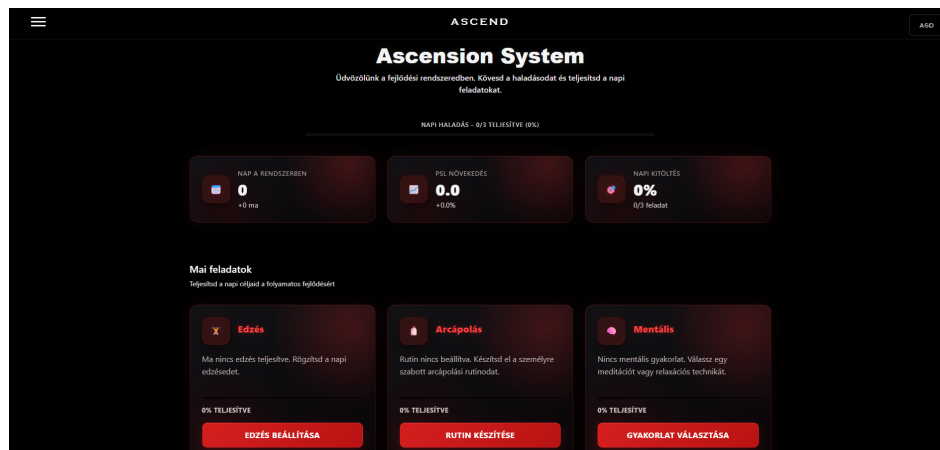
The image displays two overlapping user interface forms for login and registration. The forms have a dark, almost black, background with light gray text and red buttons. The left form is titled 'REGISZTRÁCIÓ' (Registration) and contains fields for 'FELHASZNÁLÓNÉV' (Username), 'E-MAIL CÍM' (Email Address), 'JELSZÓ' (Password), and 'JELSZÓ MEGERŐSÍTÉSE' (Confirm Password), followed by a red 'REGISZTRÁCIÓ' button. The right form is titled 'BEJELENTKEZÉS' (Login) and contains fields for 'E-MAIL VAGY FELHASZNÁLÓNÉV' (Email or Username) and 'JELSZÓ' (Password), followed by a red 'BELÉPÉS' (Login) button. Both forms have a toggle at the top to switch between 'BEJELENTKEZÉS' and 'REGISZTRÁCIÓ'.

4.1. ábra. Bejelentkezési és regisztrációs felület

A 4.1. ábrán a nyitóoldal hitelesítési modulja látható. A felhasználó itt regisztrálhat vagy bejelentkezhets, majd sikeres hitelesítés után a dashboard felületre kerül. A folyamat során a rendszer jogosultsági tokent használ a védett API hívásokhoz.

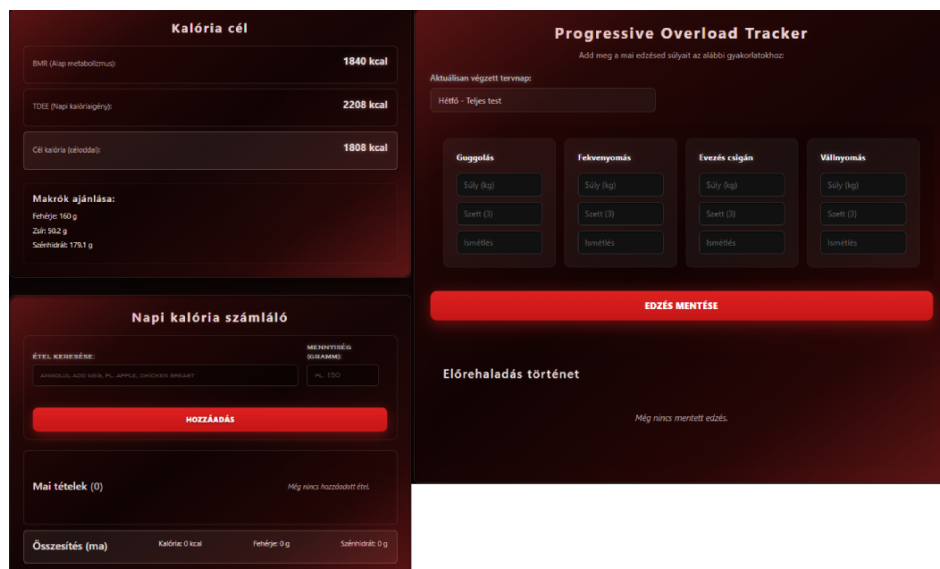
4.2. Dashboard áttekintő

A 4.2. ábra a fő áttekintő oldalt mutatja, ahol összesített visszajelzés jelenik meg a felhasználói aktivitásról. Innen közvetlenül elérhetők a fő modulok. A nézet célja az átlátható napi állapotkövetés.



4.2. ábra. Dashboard nézet

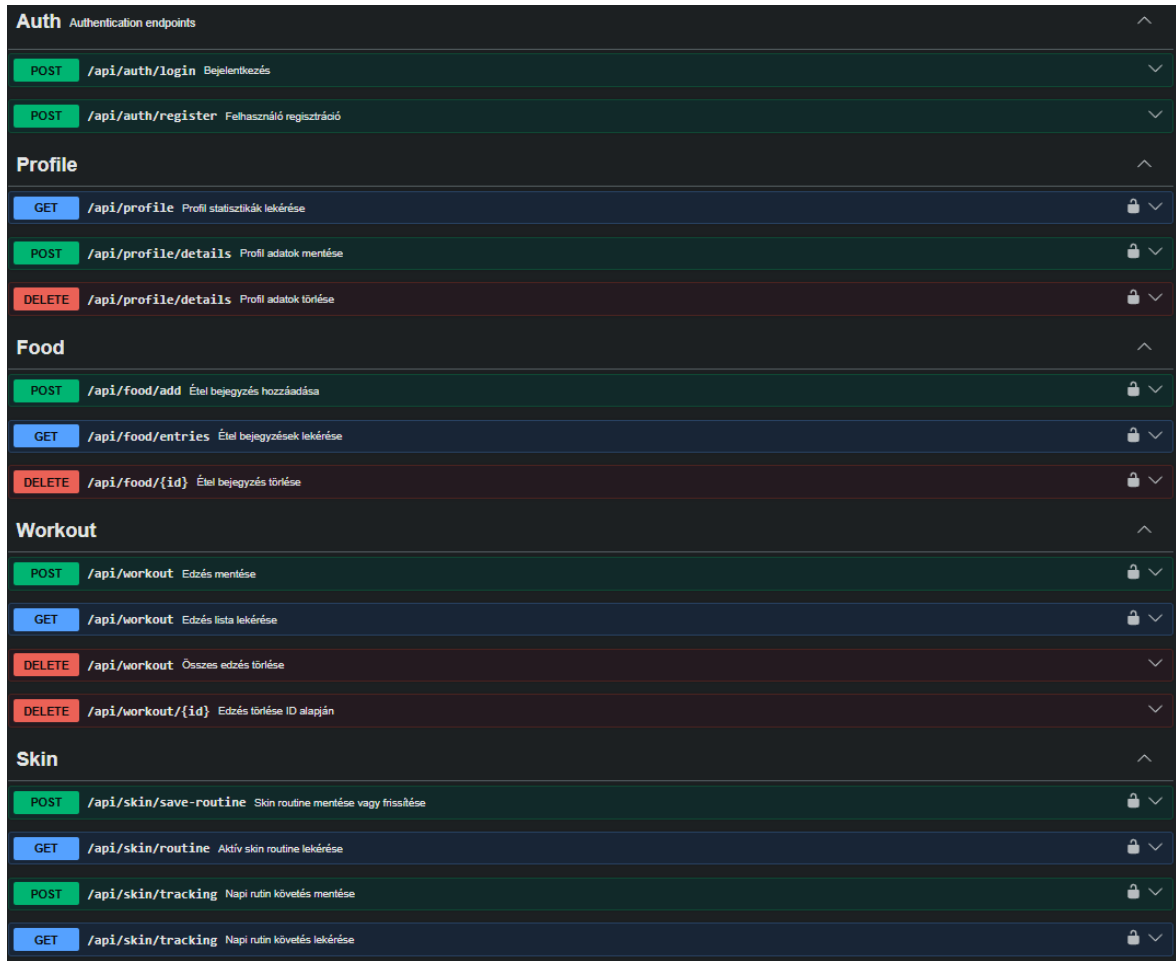
4.3. Test modul: étkezés és edzés



4.3. ábra. Étkezési és edzésbejegyzések kezelése

A 4.3. ábra a napi étel- és edzésadatok rögzítésének felületét mutatja. A mentett adatok azonnali visszajelzést adnak a felhasználó aktuális állapotáról. A modul a rendszeres nyomon követést támogatja.

4.4. API dokumentáció (Swagger)



4.4. ábra. Swagger API dokumentáció

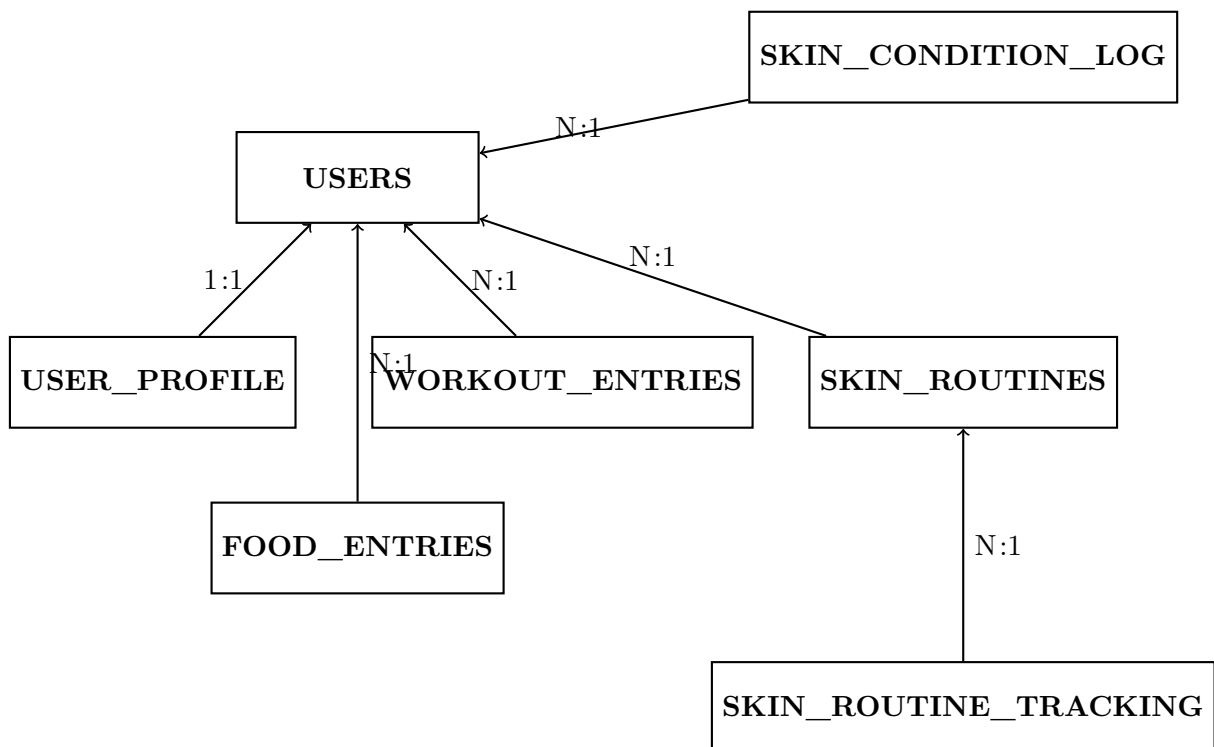
A 4.4. ábra a backend végpontok interaktív dokumentációját mutatja. Ezen keresztül ellenőrizhető a kérés-válasz szerkezet és a végpontok működése. Ez gyorsítja a hibakeresést és az integrációs ellenőrzést.

5. fejezet

Rendszertervezés

5.1. Adatmodell és architektúráis kapcsolatok

Az Ascension alkalmazás adatbázisa (`ascension_db`) a felhasználói azonosítás, profil-kezelés, táplálkozási napló, edzésnapló és arcápolási követés fő entitásait tartalmazza. A rendszer központi eleme a felhasználó (`users`), amelyhez egy-egy vagy egy-több kapcsolatban csatlakoznak a profil- és naplózási táblák.



A kapcsolatok idegen kulcsokra épülnek, a naplózó táblákon pedig felhasználó- és dátumalapú indexek támogatják a gyors lekérdezést. Az adatmodell célja a konzisztens működés fenntartása úgy, hogy közben a bővíthetőség is megmaradjon.

5.2. REST API tervezési összefoglaló

Az Ascension backend REST API-ja a hitelesítés, profilkezelés, étel- és edzésnaplózás, valamint az arcápolási modul műveleteit biztosítja. A tervezés erőforrás-alapú szemléletet követ, a védett műveletek JWT tokenhez kötöttek. A kérés-válasz struktúrák dokumentálása Swagger felületen történik.

A tervezési cél az volt, hogy a kliensoldali folyamatokat közvetlen, kiszámítható API-műveletek támogassák. A végpontok többsége CRUD mintát követ, a validáció és jogosultságellenőrzés szerveroldalon történik, így a hibakezelés egységes marad.

5.3. Felületi és navigációs tervezési elvek

A frontend React Router alapú oldalszerkezetet használ, ahol a fő útvonalak a nyitóoldal, a dashboard és a funkciómodulok. A felületi tervezés célja az átlátható, gyorsan tanulható használat volt, ezért a fő műveletek közvetlenül elérhetők maradtak.

A vizuális kialakításban kontrasztos, fókuszvezérelt megjelenítés szerepel, kártya-alapú elrendezéssel és visszajelző elemekkel. A mobil és asztali használhatóság fenntartása érdekében a komponensek töréspontokra optimalizáltan készültek.

5.4. Tervezési döntések indoklása

A rendszertervezés során a legfontosabb szempont a moduláris felépítés volt. A kliens, szerver és adatkezelési réteg szétválasztása csökkenti a komponensek közti függőséget, ami karbantarthatósági és hibakeresési előnyt ad.

A választott technológiai és szerkezeti megoldások célja az volt, hogy a rendszer a jelenlegi funkciók mellett később is bővíthető maradjon. Ezt támogatja a dokumentált API-réteg, a konzisztens adatmodell és a fejlesztés közben alkalmazott egységes ellenőrzési folyamat.

6. fejezet

Megvalósítás

6.1. Adatbázis réteg implementációja

Az Ascension rendszer adatperzisztenciáját MySQL alapú relációs adatbázis biztosítja (`ascension_db`), míg a backend oldali adatkezelést Sequelize ORM valósítja meg. A kapcsolat induláskor ellenőrzésre kerül, így a hibás konfigurációk és elérhetőségi problémák már a futtatás elején azonosíthatók.

```
const sequelize = new Sequelize("ascension_db", "root", "", {
  host: "localhost",
  dialect: "mysql",
  logging: false,
});
```

Az adatmodell felhasználó-központú szerkezetű: a `users` entitáshoz kapcsolódik a profilkezelés, az étkezési és edzésnapló, valamint az arcápolási rutin és követés. A kapcsolatok idegen kulcsokra épülnek, a naplózó tábláknál felhasználó- és dátumalapú indexelés támogatja a gyors lekérdezést.

Komponens	Szerep
<code>users</code> , <code>user_profile</code>	Hitelesítés és személyes profiladatok tárolása
<code>food_entries</code>	Napi étkezési bejegyzések és tápérték-adatok kezelése
<code>workout_entries</code>	Edzésadatok (időtartam, terhelés, jegyzetek) rögzítése
<code>skin_routines</code> , <code>skin_routine_tracking</code>	Arcápolási rutin mentése és napi követése
Indexelés (<code>user_id</code> + <code>date</code>)	Gyors napi/heti lekérdezések támogatása
Integritási szabályok	Idegen kulcsok és <code>ON DELETE CASCADE</code> konzisztens adathoz

6.2. Backend implementáció és üzleti logika

A backend Express.js alapú REST szolgáltatásként működik a `localhost:3000` címen. A fő funkcióterületek: autentikáció, profilkezelés, étel- és edzésnaplózás, valamint arc-ápolási műveletek. A klienssel való kommunikáció végpont-alapon történik, a működés és kérés-válasz szerkezet Swagger felületen dokumentált.

Az üzleti logika moduláris felépítésű, így az egyes funkciók elkülönítve fejleszthetők és tesztelhetők. A szerveroldali validáció biztosítja, hogy hibás vagy hiányos bemenet esetén is egységes válaszszerkezet maradjon.

```
const user = await User.findOne({
  where: {
    [Op.or]: [{ email: emailOrUsername }, { username: emailOrUsername }]
  }
});
```

6.3. Biztonság, jogosultság és hibakezelés

A hitelesítés JWT alapú, a jelszavak bcrypt hash formában tárolódnak. A védett végpontok kizárólag érvényes tokennel érhetők el, így a felhasználói adatok hozzáférése jogosultsághoz kötött marad.

A hibakezelés célja a konzisztens, diagnosztizálható működés: a rendszer strukturált JSON válaszokat és egységes HTTP státuszkódokat használ (200, 400, 401, 403, 404, 500). Ez támogatja mind a kliensoldali feldolgozást, mind a fejlesztői hibakeresést.

A konfiguráció és telepítés során alkalmazott döntések célja az volt, hogy a projekt lokálisan gyorsan reprodukálható maradjon, miközben a későbbi éles környezetre történő átállás technikailag előkészített legyen.

7. fejezet

Minősegbiztosítás és Tesztelés

7.1. Unit tesztek (Egységtesztelés)

A projektben valós unit teszt készült a backend edzés-kalória becslési logikájára. A teszt célja annak ellenőrzése, hogy a MET alapú számítás következetesen és hibamentesen működik különböző bemeneti eseteknél.

A teszt az alábbi területeket ellenőrzi:

- az edzéstípushoz tartozó MET értékek helyes hozzárendelése,
- a kalóriaszámítás determinisztikus eredménye normál bemenetre,
- a szélsőséges bemenetek kezelése (minimum/maximum clamp és alapértelmezett értékek).

A futtatás parancsa:

```
1 npm run test:unit
```

A futtatás eredménye:

- tesztek száma: 4
- sikeres: 4
- sikertelen: 0

7.2. API tesztelés (Postman/Swagger)

Az API végpontok ellenőrzése Swagger felületen végzett manuális próbákkal, valamint automatizált integrációs tesztekkel történt. A teljes körű futás adatbázis-elérhetőség mellett végezhető el, ezért a teszteredmények környezetfüggők lehetnek.

A futtatás parancsa:

```
1 npm run test:api
```

Teszteset	Elvárt eredmény	Eredmény
Regisztráció érvényes adatokkal	Sikeres mentés + token/azonosító	Megfelelt
Bejelentkezés hibás jelszóval	401 válaszkód	Megfelelt
Bejelentkezés helyes adatokkal	Érvényes JWT token	Megfelelt
Profil mentése token nélkül	401/403 válaszkód	Megfelelt
Étel bejegyzés létrehozása	Sikeres mentés	Megfelelt
Edzés létrehozás + törlés	Sikeres létrehozás és törlés	Megfelelt

7.3. Felhasználói felület és Reszponzivitás tesztelése

A felhasználói felület ellenőrzése elsődlegesen manuális funkcionális teszteléssel történt. A fő folyamatokat vizsgáltuk: hitelesítés, dashboard elérés, profiladat mentés, étkezési és edzésnapló kezelés.

A jelenlegi dokumentált állapotban több felbontásra és eszköztípusra kiterjedő, formálisan rögzített rezponzivitási tesztjegyzőkönyv még nem készült. Emiatt a rezponzív működésre vonatkozó megállapítások tájékoztató jellegűek, nem teljes körű validáció eredményei.

A további fejlesztési lépések között szerepel a külön asztali, tablet és mobil nézetekhez tartozó tesztesetek részletes dokumentálása.

7.4. Hibajegyzék és javítások

A fejlesztés során felmerült hibák többsége konfigurációs, validációs vagy jogosultsági jellegű volt. A javítások után az érintett funkciók újraellenőrzése megtörtént.

Hibatípus	Leírás	Javítás
CORS hiba	Frontend kérés blokkolása nem engedélyezett origin miatt	Origin lista pontosítása backend oldalon
Token hiba	Védett végpont hívása hiányzó tokennel	Egységes auth ellenőrzés minden védett útvonalon
Validációs hiba	Hiányos bemenet esetén pontatlan visszajelzés	Kötelező mezőellenőrzés és strukturált hibaüzenet

8. fejezet

Felhasználói útmutató

8.1. Rendszerhasználat előfeltételei

A fejlesztői telepítés és indítás részletes technikai lépéseit a Fejlesztői környezet fejezet tartalmazza. Jelen fejezet célja a végfelhasználói működés bemutatása, ezért itt a használati folyamatokra és az egyes modulok gyakorlati alkalmazására fókuszálunk.

A rendszer használatához három futó komponens szükséges: backend szolgáltatás, elérhető adatbázis, valamint frontend kliens. Ha ezek közül bármelyik nem elérhető, a felület részben vagy teljesen működésképtelen lehet, ezért első lépésben mindig a szolgáltatások állapotának ellenőrzése javasolt.

8.2. Regisztráció és bejelentkezés

A hitelesítés a nyitóoldalon, modális felületen történik. A rendszer új felhasználó esetén regisztrációt, meglévő fiók esetén bejelentkezést biztosít.

Regisztráció menete ♦

1. A felhasználó megnyitja a regisztrációs nézetet.
2. Kitölti a kötelező mezőket: felhasználónév, e-mail cím, jelszó, jelszó megerősítése.
3. A sikeres mentés után a rendszer visszajelzést ad, majd bejelentkezett állapotot biztosít.

Bejelentkezés menete ♦

1. A felhasználó megadja az e-mail címet vagy felhasználónevet és a jelszót.
2. Sikeres hitelesítés után a rendszer a dashboard oldalra irányít.

A hitelesítés JWT alapú, a védett API-hívások Bearer tokennel történnek.

8.3. Modulok használata

Test modul ♦ A Test modul a személyes adatok, az étkezési adatok és az edzésnapló kezelését egyesíti. A felhasználó megadhatja alap paramétereit, majd ezek alapján a rendszer becsült célértékeket és napi visszajelzést ad.

Étkezés rögzítése ♦

1. Étél kiválasztása vagy keresése.
2. Mennyiség megadása grammban.
3. Mentés után a napi lista és az összesítések frissülnek.

Edzés rögzítése ♦

1. Edzéstípus vagy gyakorlat kiválasztása.
2. Időtartam, illetve opcionálisan szett, ismétlés, súly megadása.
3. Mentés után a rendszer frissíti a napi teljesítési adatokat.

Arc modul ♦ Az Arc modul jelenlegi verziója alapfunkciós felület, amely a rutinkezelési és követési bővítésekhez biztosít szerkezeti alapot.

Mentál modul ♦ A Mentál modul a jelenlegi implementációban előkészített nézet, amely későbbi mentálhigiénés funkciók integrálására szolgál.

8.4. Dashboard és profil

A dashboard a felhasználó központi áttekintő felülete. Itt jelennek meg a napi aktivitáshoz és teljesítéshez kapcsolódó összesítések, valamint a fő modulok gyors elérései.

A profilnézetben a felhasználó saját adatait kezelheti, módosíthatja, és nyomon követheti a korábbi mentések eredményét. A rendszer célja, hogy az adatok rögzítése és ellenőrzése minimális lépésből, átláthatóan történjen.

8.5. Tipikus felhasználói folyamat

1. Regisztráció vagy bejelentkezés.
2. Profil alapadatok megadása.
3. Napi étkezési és edzésadatok rögzítése.

4. Dashboard visszajelzések ellenőrzése.
5. Következő napon a naplózás folytatása, trendek figyelése.

8.6. Felhasználói hibák és rövid megoldások

A gyakorlatban leggyakrabban hitelesítési vagy hálózati jellegű hibák fordulnak elő. Érvénytelen belépési adat esetén új hitelesítés szükséges, üres vagy hiányos mezők esetén a kötelező adatok pótlása, betöltési hiba esetén pedig a kapcsolat és a futó szolgáltatások ellenőrzése javasolt.

A felhasználói útmutató célja, hogy a napi használat lépései gyorsan követhetők legyenek, ezért a technikai telepítési részletek helyett a műveleti folyamatok kerültek előtérbe.

9. fejezet

Összegzés

9.1. Projekt értékelése

Az Ascension projekt célja egy olyan modern, webalapú önfejlesztő rendszer létrehozása volt, amely egységes felületen biztosítja a hitelesítést, a profilkezelést, az étkezési és edzésnaplózást, valamint az arcápolási rutinok követését.

A megvalósítás eredményeként elkészült egy React + Vite alapú kliensoldali felület és egy Express.js + Sequelize alapú backend architektúra. A rendszer JWT alapú jogosultságkezelést alkalmaz, MySQL adatbázisra épül, és Swagger dokumentációval támogatja a fejlesztési és tesztelési folyamatot.

A jelenlegi verzió stabil, bővíthető alapot ad a további fejlesztésekhez, és alkalmas a fő felhasználói folyamatok kiszolgálására.

9.2. Továbbfejlesztési lehetőségek

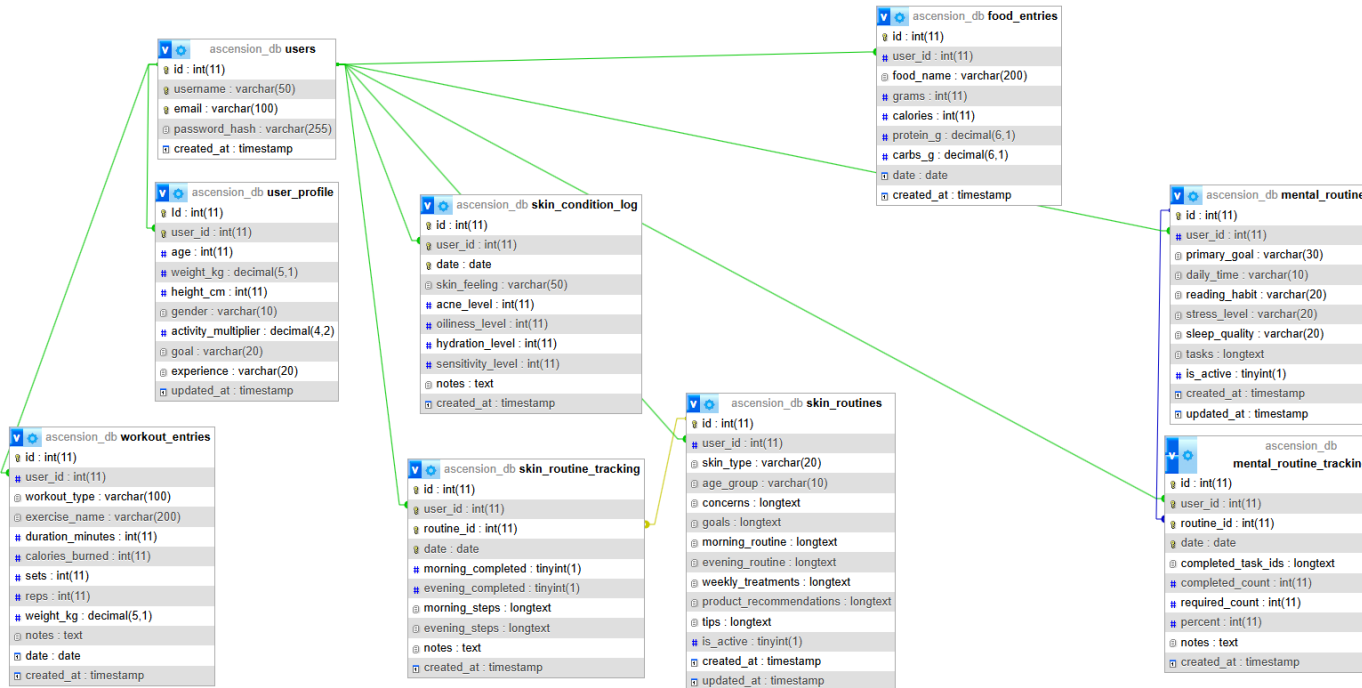
A további fejlesztés fő irányai:

- dashboard analitika és célkezelés bővítése;
- értesítési funkciók bevezetése;
- automatizált tesztelési körök kiterjesztése;
- jogosultsági és biztonsági keményítés.

Köszönetnyilvánítás

Köszönetet mondok mindazoknak, akik támogatásukkal hozzájárultak a vizsgaremek elkészítéséhez. Külön köszönöm a konzulensi iránymutatást, a szakmai visszajelzéseket, valamint a projekt tesztelésében és értékelésében nyújtott segítséget.

Mellékletek



1. ábra. Adatbázis ábra (saját készítés)

Irodalomjegyzék

- [1] Node.js Documentation, <https://nodejs.org/en/docs>, Letöltve: 2026. április.
- [2] Express.js Documentation, <https://expressjs.com>, Letöltve: 2026. április.
- [3] React Documentation, <https://react.dev>, Letöltve: 2026. április.
- [4] Vite Documentation, <https://vite.dev/guide>, Letöltve: 2026. április.
- [5] Sequelize Documentation, <https://sequelize.org/docs/v6>, Letöltve: 2026. április.
- [6] MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en>, Letöltve: 2026. április.
- [7] JSON Web Token (JWT), <https://jwt.io/introduction>, Letöltve: 2026. április.
- [8] Swagger OpenAPI Documentation, <https://swagger.io/specification>, Letöltve: 2026. április.
- [9] AscensionMain GitHub repository, <https://github.com/Loero/AscensionMain>, Letöltve: 2026. április.
- [10] Ascension Trello tábla, <https://trello.com/b/p9DooQ93/ascension>, Letöltve: 2026. április.